Witness: open-source software repository plan

On-board firmware, agent, API and CLI for the ESP32-S3 camera

Witness project · Apache-2.0
Plan rev A · October 2026

- 1. Goals
- 2. Repository layout
- 3. Runtime architecture
- 4. Storage format on the SD card
- 5. Indexing pipeline
- 6. Model adapters
- 7. Agent and tools
- 8. HTTP API (v1)
- 9. Provisioning and configuration
- 10. `witness` CLI (Python)
- 11. Testing and CI
- 12. Milestones
- 13. Contributor rules (`AGENTS.md` summary)

1. Goals

- Record continuously to microSD in a rolling buffer, and never lose recording because of indexing, queries or network trouble.
- Build a semantic index of activity with a cloud (or LAN) vision model, within a hard daily budget.
- Answer natural-language questions on the device, through a tool-calling agent, and expose the same tools through REST and MCP.
- Set up from a fresh board to a camera on the network in under ten minutes, driven by a person or by a coding agent.
- Support more than one camera board through board-support packages.

Non-goals for v1: audio, H.264, cloud video storage, face recognition, exposure to the public internet, enterprise Wi-Fi.

2. Repository layout

One monorepo, `witness-cam`, so firmware, CLI, docs and CAD release together.

```
witness-cam/
├── README.md              what it is, photos, quick start
├── AGENTS.md              rules and entry points for coding agents
├── SETUP.md               agent-readable setup guide (canonical)
├── LICENSE                Apache-2.0 (software); hardware/ and docs/ carry their own
├── firmware/              ESP-IDF 5.x project
│   ├── main/              app_main: task startup, watchdogs
│   ├── components/
│   │   ├── board/         BSPs: foriot_s3cam (default), freenove_s3, ...
│   │   ├── capture/       esp32-camera setup, frame pacing, sensor tuning
│   │   ├── recorder/      segment writer, frame index, ring-buffer eviction
│   │   ├── activity/      1/8-scale JPEG decode, motion score
│   │   ├── indexer/       keyframe policy, caption queue, JSONL index, events
│   │   ├── llm/           provider adapters, budget, retries
│   │   ├── agent/         tool registry, tool-calling loop
│   │   ├── api/           REST handlers, MJPEG and AVI streaming, auth
│   │   ├── mcp/           MCP server (Streamable HTTP, JSON-RPC 2.0)
│   │   ├── provisioning/  Improv serial, SoftAP portal, NVS config
│   │   ├── timebase/      SNTP, time zone, unsynced-segment repair
│   │   ├── marks/         robot marks over HTTP and trigger input
│   │   └── ota/           HTTPS OTA with rollback
│   ├── webui/             small static UI (Preact), packed into LittleFS
│   ├── partitions.csv
│   └── sdkconfig.defaults[.board]
├── cli/                   Python package `witness-cli` (command `witness`)
├── hardware/              CAD source, STEP, STL and 3MF from Astra; PARTS.md
├── docs/                  white paper, API (OpenAPI 3.1), index format, MCP tools
├── flasher/               ESP Web Tools page and manifest (GitHub Pages)
├── test/
│   ├── host/              recorder, index and agent logic built for Linux
│   ├── hil/               hardware-in-the-loop smoke test
│   └── eval/              labeled robot sessions for search-quality scoring
└── .github/workflows/     build per board, host tests, release, Pages
```

Why ESP-IDF rather than Arduino. The Lloyd prototype used Arduino, which was fine for one endpoint. Witness needs explicit task priorities and core pinning, the IDF HTTP server with several concurrent sockets, the SDMMC and FATFS tuning knobs, HTTPS OTA with rollback, and NVS encryption. The prototype's working camera settings (pin map, `CAMERA_GRAB_LATEST`, OV3660 white-balance warm-up) carry over directly. PlatformIO stays as an optional build entry point for contributors who already use it.

3. Runtime architecture

Task	Core	Priority	Job
`capture`	1	High	Gets frames at the configured rate and pushes them to a PSRAM ring of frame pointers

Task	Core	Priority	Job
`recorder`	1	High	Writes frames to the current segment in 64 KB chunks, appends the frame index, rotates every 60 s, evicts the oldest segments
`activity`	0	Medium	Decodes every Nth frame at 1/8 scale to grayscale, scores motion against a running background
`indexer`	0	Low	Picks keyframes, calls the model, writes index records and events
`httpd`	0	Medium	REST, MCP, live MJPEG, clip streaming
`agent`	0	Low	One `/ask` at a time; queues others

Rule: the recorder never waits on anything below it. If the SD card stalls, capture drops frames and counts them. If the indexer falls behind, keyframes are dropped by priority (watch-list candidates last).

Memory budget (8 MB PSRAM): camera buffers ~0.4 MB, frame ring ~1.5 MB, SD write buffers 0.25 MB, model request and response buffers ~0.5 MB, agent context 0.25 MB, plus mbedTLS allocations moved to PSRAM.

Flash (16 MB): two 4 MB OTA app slots, 2 MB LittleFS (web UI), NVS (64 KB, encrypted when enabled), core dump (64 KB).

4. Storage format on the SD card

Documented in `docs/storage-format.md` and versioned, so other tools can read a card directly.

```
/witness/
  card.json                          card id, format version, camera id
  rec/2026-10-29/19/
    2026-10-29T19-04-00Z.mjpg        concatenated JPEG frames, 60 s
    2026-10-29T19-04-00Z.idx         binary: u64 t_ms_utc, u32 offset, u32 len, u16 motion
  index/2026-10-29/19.jsonl          one keyframe record per line
  events/2026-10-29.jsonl            merged events with start and end
  marks/2026-10-29.jsonl             robot marks
  vec/2026-10-29/19.vec              optional int8 caption embeddings (phase 2)
```

Index record:

```
{"t":"2026-10-29T19:07:41.200Z","seg":"2026-10-29T19-07-00Z","frame":206,
 "motion":0.41,"caption":"Pipette head lowers onto tip rack column 3",
 "objects":["pipette head","tip rack"],"actions":["tip pickup"],
 "robot_state":"moving","watch":{"tip pickup":0.92,"person in frame":0.02},
 "model":"<provider>/<model>","verified":false}
```

Eviction: when free space drops below `storage.reserve_pct` (default 10%), delete the oldest hour directory under `rec/`. Delete its `index/` and `vec/` files only once they are older than `storage.index_keep_days` (default 30). Index text is tiny, so a camera can still answer "what happened last week" from captions after that week's video is gone.

Card: FAT32, 8–128 GB, high-endurance cards recommended. Mount in 1-bit SDMMC mode on CLK 39, CMD 38, D0 40. Call `setPins` before `begin` to avoid the PSRAM pin overlap.

5. Indexing pipeline

1. **Activity.** A motion score from 0 to 1 per sampled frame.
2. **Keyframe policy.** One idle keyframe every `index.idle_s` (60 s). While motion stays above `index.motion_threshold`, one keyframe every `index.active_s` (3 s). Always take a keyframe at a robot mark.
3. **Batching.** Up to four keyframes per request, each labeled with its timestamp, to cut per-request overhead.
4. **Prompt.** A system prompt plus the user's scene profile and watch list. The response must follow a JSON schema; invalid JSON is retried once and then dropped.
5. **Events.** Consecutive records sharing an action, or a watch-list hit above `watch.threshold`, are merged into events with start and end times.
6. **Budget.** `budget.frames_per_day` and `budget.tokens_per_day` are hard caps. When a cap is reached, indexing pauses and `/status` reports it. Recording continues.

6. Model adapters

Adapter	Covers
`openai_compatible` (default)	Chat Completions with tools and base64 images. Works with OpenAI, OpenRouter, Gemini's OpenAI-compatible endpoint, vLLM, Ollama and LM Studio on the LAN.
`anthropic`	Messages API with tools and images

Configuration: `model.base_url`, `model.name`, `model.api_key` (write-only), `model.vision` (bool), `model.max_tokens`. The repo keeps a **tested models** table in `docs/models.md` rather than hard-coding model names. TLS uses the IDF certificate bundle; `model.ca_pem` overrides it for private servers.

7. Agent and tools

The on-board agent loop: send the question with the tool list, run the tool calls the model requests, return results, and repeat. It stops after `agent.max_steps` (8) or when `agent.max_frames` (48) images have been sent. The final answer must cite moments as `{start, end, confidence, thumb_url, clip_url}`.

Tool	Images?	Purpose
`get_status`	No	Time, sync state, storage, budget, recording state
`list_recordings(from, to)`	No	Which minutes exist
`search_index(from, to, query, k)`	No	Keyword, watch-list, mark and optional vector search over records and events
`get_marks(from, to, label?)`	No	Robot marks
`inspect_window(t, before_s, after_s, n)`	Yes	Sends `n` frames around `t` to the model
`scan(from, to, every_s)`	Yes	Bounded sampling when the index has no hits; reports cost first
`make_clip(from, to)`	No	Returns clip URLs (MJPEG stream and AVI download)
`get_live_frame`	Yes	Current view
`set_watchlist(items)` / `set_profile(text)`	No	Configuration, behind an admin scope

The MCP server exposes these same tools, except that `inspect_window` and `scan` return image content to the calling agent instead of calling the on-board model. An external agent such as Cursor can then use its own model and spend nothing on the camera's budget.

8. HTTP API (v1)

All endpoints except `/api/v1/ping` require `Authorization: Bearer <token>`. The full OpenAPI 3.1 document lives in `docs/api.yaml`.

Method and path	Returns
`GET /api/v1/ping`	`{"ok":true,"id":"witness-ab12","fw":"0.3.0"}` (no auth; used by discovery)
`GET /api/v1/status`	Wi-Fi, time sync, SD, recording, budget, version
`GET /api/v1/frame.jpg`	Current frame
`GET /api/v1/stream`	Live MJPEG (multipart)

Method and path	Returns
`GET /api/v1/recordings?from&to`	Minute list with frame counts
`GET /api/v1/frames?from&to&every_s`	JSON list of frame URLs
`GET /api/v1/frame/{t}.jpg`	The recorded frame nearest to `t`
`GET /api/v1/clip.mjpg?from&to&speed`	Replays a recorded window as MJPEG (plays in an `<img>`)
`GET /api/v1/clip.avi?from&to`	MJPEG AVI, muxed on the fly from the frame index
`GET /api/v1/index?from&to&q`	Matching index records
`GET /api/v1/events?from&to&since_id`	Events (poll for alerts)
`POST /api/v1/marks`	`{"label":"step 12: pick tip","t?":...}`
`POST /api/v1/ask`	`{"question":"..."}` → answer with cited moments
`GET` / `PATCH /api/v1/config`	Settings; secrets are write-only
`POST /api/v1/ota`	Firmware URL or upload (admin)
`POST /mcp`	MCP Streamable HTTP endpoint

Times are accepted as RFC 3339, as local wall time (`12:00`, interpreted in `time.tz` for today), or as relative times (`-30m`).

9. Provisioning and configuration

- **Serial (default, agent-friendly).** The firmware speaks the open **Improv Wi-Fi serial** protocol on the TTL UART at 115200, plus a line-based JSON console (`{"cmd":"wifi.set",...}`) for everything else. `witness provision` wraps both.
- **Browser flasher.** ESP Web Tools on GitHub Pages flashes the board and runs Improv in Chrome or Edge, with no installs.
- **SoftAP fallback.** With no stored network, or after holding BOOT for 5 s, the camera opens `Witness-Setup-AB12` with a portal at `192.168.4.1`.
- **Identity.** The hostname `witness-<last 4 hex of MAC>` is advertised over mDNS as `_http._tcp` and `_witness._tcp`, with TXT `id`, `fw` and `board`.
- **Token.** Generated at first provisioning and returned once over serial to the CLI, which stores it in the OS keychain. It can be rotated with `witness token rotate`.

Log lines use the `key=value` style from the prototype (`wifi=ip=...`, `sd=ok size_gb=29.7`, `time=synced`) so people and agents can grep them.

10. `witness` CLI (Python)

Command	Does
`witness doctor`	Checks Python, esptool, ports, drivers and ffmpeg, then prints fixes
`witness ports`	Lists serial ports and labels them TTL (CH340, VID 1a86) or OTG (VID 303a)
`witness flash [--port] [--board]`	Downloads the release, then runs esptool at 115200 on TTL
`witness provision --ssid`	Improv Wi-Fi. The password comes from `WITNESS_WIFI_PASSWORD` or a hidden prompt. Saves the token.
`witness discover`	Finds cameras over mDNS, then checks each with `/ping`
`witness status/ask/pull/export`	API wrappers; `export` produces MP4 with ffmpeg
`witness config set/get`	Config. `-` reads a secret from stdin.
`witness mcp-config`	Prints a ready `.cursor/mcp.json` entry
`witness logs`	Tails the TTL serial log

11. Testing and CI

- **Host tests (CI):** ring-buffer eviction, index merge, query parsing, AVI muxing and agent loop logic against a mocked model. These are built with the IDF `linux` target or plain CMake.
- **Firmware builds (CI):** one job per board BSP. Release artifacts are a merged factory binary, an OTA binary, the web flasher manifest and checksums.
- **Hardware-in-the-loop (HIL, manual or self-hosted runner):** `test/hil/smoke.py` flashes, provisions, waits for `time=synced`, records two minutes, then checks the frame index, clip export and `/ask` on a mocked model.
- **Search evaluation:** `test/eval/` holds recorded robot sessions with human-labeled events (tip pickup, drop, pour). Each release reports recall and timing error for the standard questions.

12. Milestones

Milestone	Scope	Exit criterion
M0	IDF skeleton, FORIOT BSP, camera, SD mount	Frames on SD with a valid index
M1	Recorder, eviction, REST, live view, clips	24 h soak with no dropped segments and eviction working

Milestone	Scope	Exit criterion
M2	Improv, SoftAP, CLI, web flasher, SETUP.md	A fresh board reaches the network by following SETUP.md, with an agent driving
M3	Activity, indexer, watch list, marks, search	Eval recall ≥ 0.8 on tip-pickup sessions at default budget
M4	Agent `/ask`, MCP server	The headline question answered correctly with clips from Cursor via MCP
M5	OTA with rollback, encrypted builds, second BSP	OTA tested on both boards
Later	Embeddings, multi-camera hub	

13. Contributor rules (`AGENTS.md` summary)

- Never commit secrets. `firmware/main/secrets.h` and `.env` are gitignored; examples ship as `*.example`.
- Keep the storage format and API versioned. Breaking changes bump `v1` to `v2`.
- Every new board adds a BSP and a board-kit CAD folder together.
- Recording comes before everything else. A pull request that can block the recorder task is rejected.